SuperKnob配置指南

在克隆远程仓库之后，platformio中导入TFT_eSPI库和lvgl库，需要对这两个库中的其中两个文件进行配置才能正常编译并使显示屏正常工作

> 参考视频https://www.bilibili.com/video/BV1jvh7z3E9W/?vd_source=bc76eeb177b6e51d565be82f8d14a9e5
>
> 视频中的屏幕和实际使用的屏幕驱驱动和尺寸不同，实际使用的屏幕为ST7789驱动，240*320像素

需要修改的配置文件

1. superknob_demo/.pio/libdeps/lolin32_lite/TFT_eSPI/User_Setup.h 编辑这个文件使之适配显示屏（ST7789驱动，240*320像素）

User_Setup.h

```
//                            USER DEFINED SETTINGS
//   Set driver type, fonts to be loaded, pins used and SPI control method etc
//
//   See the User_Setup_Select.h file if you wish to be able to define multiple
//   setups and then easily select which setup file is used by the compiler.
//
//   If this file is edited correctly then all the library example sketches should
//   run without the need to make any more changes for a particular hardware setup!
//   Note that some sketches are designed for a particular TFT pixel width/height

// User defined information reported by "Read_User_Setup" test & diagnostics example
#define USER_SETUP_INFO "User_Setup"

// Define to disable all #warnings in library (can be put in User_Setup_Select.h)
//#define DISABLE_ALL_LIBRARY_WARNINGS

// ##################################################################################
//
// Section 1. Call up the right driver file and any options for it
//
```

```
// ##################################################################################

// Define STM32 to invoke optimised processor support (only for STM32)
//#define STM32

// Defining the STM32 board allows the library to optimise the performance
// for UNO compatible "MCUfriend" style shields
//#define NUCLEO_64_TFT
//#define NUCLEO_144_TFT

// STM32 8 bit parallel only:
// If STN32 Port A or B pins 0-7 are used for 8 bit parallel data bus bits 0-7
// then this will improve rendering performance by a factor of ~8x
//#define STM_PORTA_DATA_BUS
//#define STM_PORTB_DATA_BUS

// Tell the library to use parallel mode (otherwise SPI is assumed)
//#define TFT_PARALLEL_8_BIT
//#defined TFT_PARALLEL_16_BIT // **** 16 bit parallel ONLY for RP2040 processor ****

// Display type -  only define if RPi display
//#define RPI_DISPLAY_TYPE // 20MHz maximum SPI

// Only define one driver, the other ones must be commented out
// #define ILI9341_DRIVER       // Generic driver for common displays
//#define ILI9341_2_DRIVER     // Alternative ILI9341 driver, see https://github.com/Bodmer/TFT_eSPI/issues/1172
//#define ST7735_DRIVER      // Define additional parameters below for this display
//#define ILI9163_DRIVER     // Define additional parameters below for this display
//#define S6D02A1_DRIVER
//#define RPI_ILI9486_DRIVER // 20MHz maximum SPI
//#define HX8357D_DRIVER
//#define ILI9481_DRIVER
//#define ILI9486_DRIVER
//#define ILI9488_DRIVER     // WARNING: Do not connect ILI9488 display SDO to MISO if other devices share the SPI bus (TFT SDO does NOT tristate when CS is high)
#define ST7789_DRIVER      // Full configuration option, define additional parameters below for this display
//#define ST7789_2_DRIVER    // Minimal configuration option, define additional parameters below for this display
//#define R61581_DRIVER
```

```
//#define RM68140_DRIVER
//#define ST7796_DRIVER
//#define SSD1351_DRIVER
//#define SSD1963_480_DRIVER
//#define SSD1963_800_DRIVER
//#define SSD1963_800ALT_DRIVER
//#define ILI9225_DRIVER
//#define GC9A01_DRIVER

// Some displays support SPI reads via the MISO pin, other displays have a single
// bi-directional SDA pin and the library will try to read this via the MOSI line.
// To use the SDA line for reading data from the TFT uncomment the following line:

// #define TFT_SDA_READ      // This option is for ESP32 ONLY, tested with ST7789 and GC9A01 display only

// For ST7735, ST7789 and ILI9341 ONLY, define the colour order IF the blue and red are swapped on your display
// Try ONE option at a time to find the correct colour order for your display

 #define TFT_RGB_ORDER TFT_RGB  // Colour order Red-Green-Blue
//  #define TFT_RGB_ORDER TFT_BGR  // Colour order Blue-Green-Red

// For M5Stack ESP32 module with integrated ILI9341 display ONLY, remove // in line below

// #define M5STACK

// For ST7789, ST7735, ILI9163 and GC9A01 ONLY, define the pixel width and height in portrait orientation
// #define TFT_WIDTH  80
// #define TFT_WIDTH  128
// #define TFT_WIDTH  172 // ST7789 172 x 320
#define TFT_WIDTH  240 // ST7789 240 x 240 and 240 x 320
// #define TFT_HEIGHT 160
// #define TFT_HEIGHT 128
// #define TFT_HEIGHT 240 // ST7789 240 x 240
#define TFT_HEIGHT 320 // ST7789 240 x 320
// #define TFT_HEIGHT 240 // GC9A01 240 x 240

// For ST7735 ONLY, define the type of display, originally this was based on the
// colour of the tab on the screen protector film but this is not always true, so try
```

```
// out the different options below if the screen does not display graphics correctly,
// e.g. colours wrong, mirror images, or stray pixels at the edges.
// Comment out ALL BUT ONE of these options for a ST7735 display driver, save this
// this User_Setup file, then rebuild and upload the sketch to the board again:

// #define ST7735_INITB
// #define ST7735_GREENTAB
// #define ST7735_GREENTAB2
// #define ST7735_GREENTAB3
// #define ST7735_GREENTAB128    // For 128 x 128 display
// #define ST7735_GREENTAB160x80 // For 160 x 80 display (BGR, inverted, 26 offset)
// #define ST7735_ROBOTLCD       // For some RobotLCD arduino shields (128x160, BGR, https://docs.arduino.cc/retired/getting-started-guides/TFT)
// #define ST7735_REDTAB
// #define ST7735_BLACKTAB
// #define ST7735_REDTAB160x80   // For 160 x 80 display with 24 pixel offset

// If colours are inverted (white shows as black) then uncomment one of the next
// 2 lines try both options, one of the options should correct the inversion.

// #define TFT_INVERSION_ON
// #define TFT_INVERSION_OFF

// ###################################################################################
//
// Section 2. Define the pins that are used to interface with the display here
//
// ###################################################################################

// If a backlight control signal is available then define the TFT_BL pin in Section 2
// below. The backlight will be turned ON when tft.begin() is called, but the library
// needs to know if the LEDs are ON with the pin HIGH or LOW. If the LEDs are to be
// driven with a PWM signal or turned OFF/ON then this must be handled by the user
// sketch. e.g. with digitalWrite(TFT_BL, LOW);

```

```
// #define TFT_BL   32            // LED back-light control pin
// #define TFT_BACKLIGHT_ON HIGH  // Level to turn ON back-light (HIGH or LOW)

// We must use hardware SPI, a minimum of 3 GPIO pins is needed.
// Typical setup for ESP8266 NodeMCU ESP-12 is :
//
// Display SDO/MISO  to NodeMCU pin D6 (or leave disconnected if not reading TFT)
// Display LED       to NodeMCU pin VIN (or 5V, see below)
// Display SCK       to NodeMCU pin D5
// Display SDI/MOSI  to NodeMCU pin D7
// Display DC (RS/AO)to NodeMCU pin D3
// Display RESET     to NodeMCU pin D4 (or RST, see below)
// Display CS        to NodeMCU pin D8 (or GND, see below)
// Display GND       to NodeMCU pin GND (0V)
// Display VCC       to NodeMCU 5V or 3.3V
//
// The TFT RESET pin can be connected to the NodeMCU RST pin or 3.3V to free up a control pin
//
// The DC (Data Command) pin may be labelled AO or RS (Register Select)
//
// With some displays such as the ILI9341 the TFT CS pin can be connected to GND if no more
// SPI devices (e.g. an SD Card) are connected, in this case comment out the #define TFT_CS
// line below so it is NOT defined. Other displays such at the ST7735 require the TFT CS pin
// to be toggled during setup, so in these cases the TFT_CS line must be defined and connected.
//
// The NodeMCU D0 pin can be used for RST
//
//
// Note: only some versions of the NodeMCU provide the USB 5V on the VIN pin
// If 5V is not available at a pin you can use 3.3V but backlight brightness
// will be lower.

// ###### EDIT THE PIN NUMBERS IN THE LINES FOLLOWING TO SUIT YOUR ESP8266 SETUP ######

// For NodeMCU - use pin numbers in the form PIN_Dx where Dx is the NodeMCU pin designation
// #define TFT_CS   PIN_D8  // Chip select control pin D8
// #define TFT_DC   PIN_D3  // Data Command control pin
// #define TFT_RST  PIN_D4  // Reset pin (could connect to NodeMCU RST, see next line)
```

```
//#define TFT_RST  -1    // Set TFT_RST to -1 if the display RESET is connected to NodeMCU RST or 3.3V

//#define TFT_BL PIN_D1  // LED back-light (only for ST7789 with backlight control pin)

//#define TOUCH_CS PIN_D2     // Chip select pin (T_CS) of touch screen

//#define TFT_WR PIN_D2       // Write strobe for modified Raspberry Pi TFT only

// ######  FOR ESP8266 OVERLAP MODE EDIT THE PIN NUMBERS IN THE FOLLOWING LINES  ######

// Overlap mode shares the ESP8266 FLASH SPI bus with the TFT so has a performance impact
// but saves pins for other functions. It is best not to connect MISO as some displays
// do not tristate that line when chip select is high!
// Note: Only one SPI device can share the FLASH SPI lines, so a SPI touch controller
// cannot be connected as well to the same SPI signals.
// On NodeMCU 1.0 SD0=MISO, SD1=MOSI, CLK=SCLK to connect to TFT in overlap mode
// On NodeMCU V3  S0 =MISO, S1 =MOSI, S2 =SCLK
// In ESP8266 overlap mode the following must be defined

//#define TFT_SPI_OVERLAP

// In ESP8266 overlap mode the TFT chip select MUST connect to pin D3
//#define TFT_CS   PIN_D3
//#define TFT_DC   PIN_D5  // Data Command control pin
//#define TFT_RST  PIN_D4  // Reset pin (could connect to NodeMCU RST, see next line)
//#define TFT_RST  -1  // Set TFT_RST to -1 if the display RESET is connected to NodeMCU RST or 3.3V

// ###### EDIT THE PIN NUMBERS IN THE LINES FOLLOWING TO SUIT YOUR ESP32 SETUP   ######

// For ESP32 Dev board (only tested with ILI9341 display)
// The hardware SPI can be mapped to any pins

//#define TFT_MISO 19
//#define TFT_MOSI 23
//#define TFT_SCLK 18
//#define TFT_CS   15  // Chip select control pin
```

```
//#define TFT_DC    2  // Data Command control pin
//#define TFT_RST   4  // Reset pin (could connect to RST pin)
//#define TFT_RST  -1  // Set TFT_RST to -1 if display RESET is connected to ESP32 board RST

// For ESP32 Dev board (only tested with GC9A01 display)
// The hardware SPI can be mapped to any pins

#define TFT_MOSI 19 // In some display driver board, it might be written as "SDA" and so on.
#define TFT_SCLK 18
#define TFT_CS   16  // Chip select control pin
#define TFT_DC   15  // Data Command control pin
#define TFT_RST  13  // Reset pin (could connect to Arduino RESET pin)
#define TFT_BL   17 // LED back-light

//#define TOUCH_CS 21     // Chip select pin (T_CS) of touch screen

//#define TFT_WR 22    // Write strobe for modified Raspberry Pi TFT only

// For the M5Stack module use these #define lines
//#define TFT_MISO 19
//#define TFT_MOSI 23
//#define TFT_SCLK 18
//#define TFT_CS   14  // Chip select control pin
//#define TFT_DC   27  // Data Command control pin
//#define TFT_RST  33  // Reset pin (could connect to Arduino RESET pin)
//#define TFT_BL   32  // LED back-light (required for M5Stack)

// ######       EDIT THE PINs BELOW TO SUIT YOUR ESP32 PARALLEL TFT SETUP
  ######

// The library supports 8 bit parallel TFTs with the ESP32, the pin
// selection below is compatible with ESP32 boards in UNO format.
// Wemos D32 boards need to be modified, see diagram in Tools folder.
// Only ILI9481 and ILI9341 based displays have been tested!

// Parallel bus is only supported for the STM32 and ESP32
// Example below is for ESP32 Parallel interface with UNO displays

// Tell the library to use 8 bit parallel mode (otherwise SPI is assumed)
//#define TFT_PARALLEL_8_BIT

// The ESP32 and TFT the pins used for testing are:
//#define TFT_CS   33  // Chip select control pin (library pulls permanently low
```

```
//#define TFT_DC   15  // Data Command control pin - must use a pin in the range 0-31
//#define TFT_RST  32  // Reset pin, toggles on startup

//#define TFT_WR    4  // Write strobe control pin - must use a pin in the range 0-31
//#define TFT_RD    2  // Read strobe control pin

//#define TFT_D0   12  // Must use pins in the range 0-31 for the data bus
//#define TFT_D1   13  // so a single register write sets/clears all bits.
//#define TFT_D2   26  // Pins can be randomly assigned, this does not affect
//#define TFT_D3   25  // TFT screen update performance.
//#define TFT_D4   17
//#define TFT_D5   16
//#define TFT_D6   27
//#define TFT_D7   14

// ######       EDIT THE PINs BELOW TO SUIT YOUR STM32 SPI TFT SETUP        ######

// The TFT can be connected to SPI port 1 or 2
//#define TFT_SPI_PORT 1 // SPI port 1 maximum clock rate is 55MHz
//#define TFT_MOSI PA7
//#define TFT_MISO PA6
//#define TFT_SCLK PA5

//#define TFT_SPI_PORT 2 // SPI port 2 maximum clock rate is 27MHz
//#define TFT_MOSI PB15
//#define TFT_MISO PB14
//#define TFT_SCLK PB13

// Can use Ardiuno pin references, arbitrary allocation, TFT_eSPI controls chip select
//#define TFT_CS   D5 // Chip select control pin to TFT CS
//#define TFT_DC   D6 // Data Command control pin to TFT DC (may be labelled RS = Register Select)
//#define TFT_RST  D7 // Reset pin to TFT RST (or RESET)
// OR alternatively, we can use STM32 port reference names PXnn
//#define TFT_CS   PE11 // Nucleo-F767ZI equivalent of D5
//#define TFT_DC   PE9  // Nucleo-F767ZI equivalent of D6
//#define TFT_RST  PF13 // Nucleo-F767ZI equivalent of D7

//#define TFT_RST  -1   // Set TFT_RST to -1 if the display RESET is connected to processor reset
                        // Use an Arduino pin for initial testing as connecting to processor reset
                        // may not work (pulse too short at power up?)
```

```
// ##################################################################################
//
// Section 3. Define the fonts that are to be used here
//
// ##################################################################################

// Comment out the #defines below with // to stop that font being loaded
// The ESP8366 and ESP32 have plenty of memory so commenting out fonts is not
// normally necessary. If all fonts are loaded the extra FLASH space required is
// about 17Kbytes. To save FLASH space only enable the fonts you need!

#define LOAD_GLCD   // Font 1. Original Adafruit 8 pixel font needs ~1820 bytes in FLASH
#define LOAD_FONT2  // Font 2. Small 16 pixel high font, needs ~3534 bytes in FLASH, 96 characters
#define LOAD_FONT4  // Font 4. Medium 26 pixel high font, needs ~5848 bytes in FLASH, 96 characters
#define LOAD_FONT6  // Font 6. Large 48 pixel font, needs ~2666 bytes in FLASH, only characters 1234567890:-.apm
#define LOAD_FONT7  // Font 7. 7 segment 48 pixel font, needs ~2438 bytes in FLASH, only characters 1234567890:-.
#define LOAD_FONT8  // Font 8. Large 75 pixel font needs ~3256 bytes in FLASH, only characters 1234567890:-.
//#define LOAD_FONT8N // Font 8. Alternative to Font 8 above, slightly narrower, so 3 digits fit a 160 pixel TFT
#define LOAD_GFXFF  // FreeFonts. Include access to the 48 Adafruit_GFX free fonts FF1 to FF48 and custom fonts

// Comment out the #define below to stop the SPIFFS filing system and smooth font code being loaded
// this will save ~20kbytes of FLASH
#define SMOOTH_FONT

// ##################################################################################
//
// Section 4. Other options
//
// ###############################################################################
```

```
###

// For RP2040 processor and SPI displays, uncomment the following line to use the PIO interface.
//#define RP2040_PIO_SPI // Leave commented out to use standard RP2040 SPI port interface

// For RP2040 processor and 8 or 16 bit parallel displays:
// The parallel interface write cycle period is derived from a division of the CPU clock
// speed so scales with the processor clock. This means that the divider ratio may need
// to be increased when overclocking. I may also need to be adjusted dependant on the
// display controller type (ILI94341, HX8357C etc). If RP2040_PIO_CLK_DIV is not defined
// the library will set default values which may not suit your display.
// The display controller data sheet will specify the minimum write cycle period. The
// controllers often work reliably for shorter periods, however if the period is too short
// the display may not initialise or graphics will become corrupted.
// PIO write cycle frequency = (CPU clock/(4 * RP2040_PIO_CLK_DIV))
//#define RP2040_PIO_CLK_DIV 1 // 32ns write cycle at 125MHz CPU clock
//#define RP2040_PIO_CLK_DIV 2 // 64ns write cycle at 125MHz CPU clock
//#define RP2040_PIO_CLK_DIV 3 // 96ns write cycle at 125MHz CPU clock

// For the RP2040 processor define the SPI port channel used (default 0 if undefined)
//#define TFT_SPI_PORT 1 // Set to 0 if SPI0 pins are used, or 1 if spi1 pins used

// For the STM32 processor define the SPI port channel used (default 1 if undefined)
//#define TFT_SPI_PORT 2 // Set to 1 for SPI port 1, or 2 for SPI port 2

// Define the SPI clock frequency, this affects the graphics rendering speed. Too
// fast and the TFT driver will not keep up and display corruption appears.
// With an ILI9341 display 40MHz works OK, 80MHz sometimes fails
// With a ST7735 display more than 27MHz may not work (spurious pixels and lines)
// With an ILI9163 display 27 MHz works OK.

// #define SPI_FREQUENCY   1000000
// #define SPI_FREQUENCY   5000000
// #define SPI_FREQUENCY  10000000
```

```
// #define SPI_FREQUENCY  20000000
// #define SPI_FREQUENCY  27000000
// #define SPI_FREQUENCY  40000000
// #define SPI_FREQUENCY  55000000 // STM32 SPI1 only (SPI2 maximum is 27MHz)
#define SPI_FREQUENCY  80000000

// Optional reduced SPI frequency for reading TFT
#define SPI_READ_FREQUENCY  20000000

// The XPT2046 requires a lower SPI clock rate of 2.5MHz so we define that here:
#define SPI_TOUCH_FREQUENCY  2500000

// The ESP32 has 2 free SPI ports i.e. VSPI and HSPI, the VSPI is the default.
// If the VSPI port is in use and pins are not accessible (e.g. TTGO T-Beam)
// then uncomment the following line:
//#define USE_HSPI_PORT

// Comment out the following #define if "SPI Transactions" do not need to be
// supported. When commented out the code size will be smaller and sketches will
// run slightly faster, so leave it commented out unless you need it!

// Transaction support is needed to work with SD library but not needed with TFT_SdFat
// Transaction support is required if other SPI devices are connected.

// Transactions are automatically enabled by the library for an ESP32 (to use HAL mutex)
// so changing it here has no effect

// #define SUPPORT_TRANSACTIONS

```

2. 将superknob_demo/.pio/libdeps/lolin32_lite/lvgl 目录下的 lv_conf_template.h文件重命名为lv_conf.h，并修改其中的部分宏定义使能lvgl的部分功能

lv_conf.h

```
/**
 * @file lv_conf.h
 * Configuration file for v8.4.0
 */

/*
 * Copy this file as `lv_conf.h`
```

```c
 * 1. simply next to the `lvgl` folder
 * 2. or any other places and
 *    - define `LV_CONF_INCLUDE_SIMPLE`
 *    - add the path as include path
 */

/* clang-format off */
#if 1 /*Set it to "1" to enable content*/

#ifndef LV_CONF_H
#define LV_CONF_H

#include <stdint.h>

/*====================
   COLOR SETTINGS
 *====================*/

/*Color depth: 1 (1 byte per pixel), 8 (RGB332), 16 (RGB565), 32 (ARGB8888)*/
#define LV_COLOR_DEPTH 16

/*Swap the 2 bytes of RGB565 color. Useful if the display has an 8-bit interface (e.g. SPI)*/
#define LV_COLOR_16_SWAP 0

/*Enable features to draw on transparent background.
 *It's required if opa, and transform_* style properties are used.
 *Can be also used if the UI is above another layer, e.g. an OSD menu or video player.*/
#define LV_COLOR_SCREEN_TRANSP 0

/* Adjust color mix functions rounding. GPUs might calculate color mix (blending) differently.
 * 0: round down, 64: round up from x.75, 128: round up from half, 192: round up from x.25, 254: round up */
#define LV_COLOR_MIX_ROUND_OFS 0

/*Images pixels with this color will not be drawn if they are chroma keyed)*/
#define LV_COLOR_CHROMA_KEY lv_color_hex(0x00ff00)         /*pure green*/

/*=========================
   MEMORY SETTINGS
 *=========================*/

/*1: use custom malloc/free, 0: use the built-in `lv_mem_alloc()` and `lv_mem_free()`*/
#define LV_MEM_CUSTOM 0
```

```c
#if LV_MEM_CUSTOM == 0
    /*Size of the memory available for `lv_mem_alloc()` in bytes (>= 2kB)*/
    #define LV_MEM_SIZE (48U * 1024U)          /*[bytes]*/

    /*Set an address for the memory pool instead of allocating it as a normal array. Can be in external SRAM too.*/
    #define LV_MEM_ADR 0     /*0: unused*/
    /*Instead of an address give a memory allocator that will be called to get a memory pool for LVGL. E.g. my_malloc*/
    #if LV_MEM_ADR == 0
        #undef LV_MEM_POOL_INCLUDE
        #undef LV_MEM_POOL_ALLOC
    #endif

#else       /*LV_MEM_CUSTOM*/
    #define LV_MEM_CUSTOM_INCLUDE <stdlib.h>   /*Header for the dynamic memory function*/
    #define LV_MEM_CUSTOM_ALLOC   malloc
    #define LV_MEM_CUSTOM_FREE    free
    #define LV_MEM_CUSTOM_REALLOC realloc
#endif     /*LV_MEM_CUSTOM*/

/*Number of the intermediate memory buffer used during rendering and other internal processing mechanisms.
 *You will see an error log message if there wasn't enough buffers. */
#define LV_MEM_BUF_MAX_NUM 16

/*Use the standard `memcpy` and `memset` instead of LVGL's own functions. (Might or might not be faster).*/
#define LV_MEMCPY_MEMSET_STD 0

/*====================
   HAL SETTINGS
 *====================*/

/*Default display refresh period. LVG will redraw changed areas with this period time*/
#define LV_DISP_DEF_REFR_PERIOD 30      /*[ms]*/

/*Input device read period in milliseconds*/
#define LV_INDEV_DEF_READ_PERIOD 30     /*[ms]*/

/*Use a custom tick source that tells the elapsed time in milliseconds.
 *It removes the need to manually update the tick with `lv_tick_inc()`)*/
#define LV_TICK_CUSTOM 1
#if LV_TICK_CUSTOM
```

```
    #define LV_TICK_CUSTOM_INCLUDE "Arduino.h"         /*Header for the system time function*/
    #define LV_TICK_CUSTOM_SYS_TIME_EXPR (millis())    /*Expression evaluating to current system time in ms*/
    /*If using lvgl as ESP32 component*/
    // #define LV_TICK_CUSTOM_INCLUDE "esp_timer.h"
    // #define LV_TICK_CUSTOM_SYS_TIME_EXPR ((esp_timer_get_time() / 1000LL))
#endif   /*LV_TICK_CUSTOM*/

/*Default Dot Per Inch. Used to initialize default sizes such as widgets sized, style paddings.
 *(Not so important, you can adjust it to modify default sizes and spaces)*/
#define LV_DPI_DEF 130     /*[px/inch]*/

/*=======================
 * FEATURE CONFIGURATION
 *=======================*/

/*-------------
 * Drawing
 *-----------*/

/*Enable complex draw engine.
 *Required to draw shadow, gradient, rounded corners, circles, arc, skew lines, image transformations or any masks*/
#define LV_DRAW_COMPLEX 1
#if LV_DRAW_COMPLEX != 0

    /*Allow buffering some shadow calculation.
    *LV_SHADOW_CACHE_SIZE is the max. shadow size to buffer, where shadow size is `shadow_width + radius`
    *Caching has LV_SHADOW_CACHE_SIZE^2 RAM cost*/
    #define LV_SHADOW_CACHE_SIZE 0

    /* Set number of maximally cached circle data.
    * The circumference of 1/4 circle are saved for anti-aliasing
    * radius * 4 bytes are used per circle (the most often used radiuses are saved)
    * 0: to disable caching */
    #define LV_CIRCLE_CACHE_SIZE 4
#endif /*LV_DRAW_COMPLEX*/

/**
 * "Simple layers" are used when a widget has `style_opa < 255` to buffer the widget into a layer
 * and blend it as an image with the given opacity.
 * Note that `bg_opa`, `text_opa` etc don't require buffering into layer)
```

```c
 * The widget can be buffered in smaller chunks to avoid using large buffers.
 *
 * - LV_LAYER_SIMPLE_BUF_SIZE: [bytes] the optimal target buffer size. LVGL will try to allocate it
 * - LV_LAYER_SIMPLE_FALLBACK_BUF_SIZE: [bytes]  used if `LV_LAYER_SIMPLE_BUF_SIZE` couldn't be allocated.
 *
 * Both buffer sizes are in bytes.
 * "Transformed layers" (where transform_angle/zoom properties are used) use larger buffers
 * and can't be drawn in chunks. So these settings affects only widgets with opacity.
 */
#define LV_LAYER_SIMPLE_BUF_SIZE          (24 * 1024)
#define LV_LAYER_SIMPLE_FALLBACK_BUF_SIZE (3 * 1024)

/*Default image cache size. Image caching keeps the images opened.
 *If only the built-in image formats are used there is no real advantage of caching. (I.e. if no new image decoder is added)
 *With complex image decoders (e.g. PNG or JPG) caching can save the continuous open/decode of images.
 *However the opened images might consume additional RAM.
 *0: to disable caching*/
#define LV_IMG_CACHE_DEF_SIZE 0

/*Number of stops allowed per gradient. Increase this to allow more stops.
 *This adds (sizeof(lv_color_t) + 1) bytes per additional stop*/
#define LV_GRADIENT_MAX_STOPS 2

/*Default gradient buffer size.
 *When LVGL calculates the gradient "maps" it can save them into a cache to avoid calculating them again.
 *LV_GRAD_CACHE_DEF_SIZE sets the size of this cache in bytes.
 *If the cache is too small the map will be allocated only while it's required for the drawing.
 *0 mean no caching.*/
#define LV_GRAD_CACHE_DEF_SIZE 0

/*Allow dithering the gradients (to achieve visual smooth color gradients on limited color depth display)
 *LV_DITHER_GRADIENT implies allocating one or two more lines of the object's rendering surface
 *The increase in memory consumption is (32 bits * object width) plus 24 bits * object width if using error diffusion */
#define LV_DITHER_GRADIENT 0
#if LV_DITHER_GRADIENT
    /*Add support for error diffusion dithering.
```

```
     *Error diffusion dithering gets a much better visual result, but implies more CPU consumption and memory when drawing.
     *The increase in memory consumption is (24 bits * object's width)*/
    #define LV_DITHER_ERROR_DIFFUSION 0
#endif

/*Maximum buffer size to allocate for rotation.
 *Only used if software rotation is enabled in the display driver.*/
#define LV_DISP_ROT_MAX_BUF (10*1024)

/*-------------
 * GPU
 *-----------*/

/*Use Arm's 2D acceleration library Arm-2D */
#define LV_USE_GPU_ARM2D 0

/*Use STM32's DMA2D (aka Chrom Art) GPU*/
#define LV_USE_GPU_STM32_DMA2D 0
#if LV_USE_GPU_STM32_DMA2D
    /*Must be defined to include path of CMSIS header of target processor
    e.g. "stm32f7xx.h" or "stm32f4xx.h"*/
    #define LV_GPU_DMA2D_CMSIS_INCLUDE
#endif

/*Enable RA6M3 G2D GPU*/
#define LV_USE_GPU_RA6M3_G2D 0
#if LV_USE_GPU_RA6M3_G2D
    /*include path of target processor
    e.g. "hal_data.h"*/
    #define LV_GPU_RA6M3_G2D_INCLUDE "hal_data.h"
#endif

/*Use SWM341's DMA2D GPU*/
#define LV_USE_GPU_SWM341_DMA2D 0
#if LV_USE_GPU_SWM341_DMA2D
    #define LV_GPU_SWM341_DMA2D_INCLUDE "SWM341.h"
#endif

/*Use NXP's PXP GPU iMX RTxxx platforms*/
#define LV_USE_GPU_NXP_PXP 0
#if LV_USE_GPU_NXP_PXP
    /*1: Add default bare metal and FreeRTOS interrupt handling routines for PXP (lv_gpu_nxp_pxp_osa.c)
    *   and call lv_gpu_nxp_pxp_init() automatically during lv_init(). Note that symbol SDK_OS_FREE_RTOS
```

```
    *   has to be defined in order to use FreeRTOS OSA, otherwise bare-metal implementation is selected.
    *0: lv_gpu_nxp_pxp_init() has to be called manually before lv_init()
    */
    #define LV_USE_GPU_NXP_PXP_AUTO_INIT 0
#endif

/*Use NXP's VG-Lite GPU iMX RTxxx platforms*/
#define LV_USE_GPU_NXP_VG_LITE 0

/*Use SDL renderer API*/
#define LV_USE_GPU_SDL 0
#if LV_USE_GPU_SDL
    #define LV_GPU_SDL_INCLUDE_PATH <SDL2/SDL.h>
    /*Texture cache size, 8MB by default*/
    #define LV_GPU_SDL_LRU_SIZE (1024 * 1024 * 8)
    /*Custom blend mode for mask drawing, disable if you need to link with older SDL2 lib*/
    #define LV_GPU_SDL_CUSTOM_BLEND_MODE (SDL_VERSION_ATLEAST(2, 0, 6))
#endif

/*-------------
 * Logging
 *-----------*/

/*Enable the log module*/
#define LV_USE_LOG 0
#if LV_USE_LOG

    /*How important log should be added:
    *LV_LOG_LEVEL_TRACE       A lot of logs to give detailed information
    *LV_LOG_LEVEL_INFO        Log important events
    *LV_LOG_LEVEL_WARN        Log if something unwanted happened but didn't cause a problem
    *LV_LOG_LEVEL_ERROR       Only critical issue, when the system may fail
    *LV_LOG_LEVEL_USER        Only logs added by the user
    *LV_LOG_LEVEL_NONE        Do not log anything*/
    #define LV_LOG_LEVEL LV_LOG_LEVEL_WARN

    /*1: Print the log with 'printf';
    *0: User need to register a callback with `lv_log_register_print_cb()`*/
    #define LV_LOG_PRINTF 0

    /*Enable/disable LV_LOG_TRACE in modules that produces a huge number of logs*/
    #define LV_LOG_TRACE_MEM        1
    #define LV_LOG_TRACE_TIMER      1
```

```
    #define LV_LOG_TRACE_INDEV      1
    #define LV_LOG_TRACE_DISP_REFR  1
    #define LV_LOG_TRACE_EVENT      1
    #define LV_LOG_TRACE_OBJ_CREATE 1
    #define LV_LOG_TRACE_LAYOUT     1
    #define LV_LOG_TRACE_ANIM       1

#endif  /*LV_USE_LOG*/

/*-------------
 * Asserts
 *-----------*/

/*Enable asserts if an operation is failed or an invalid data is found.
 *If LV_USE_LOG is enabled an error message will be printed on failure*/
#define LV_USE_ASSERT_NULL          1   /*Check if the parameter is NULL. (Very fast, recommended)*/
#define LV_USE_ASSERT_MALLOC        1   /*Checks is the memory is successfully allocated or no. (Very fast, recommended)*/
#define LV_USE_ASSERT_STYLE         0   /*Check if the styles are properly initialized. (Very fast, recommended)*/
#define LV_USE_ASSERT_MEM_INTEGRITY 0   /*Check the integrity of `lv_mem` after critical operations. (Slow)*/
#define LV_USE_ASSERT_OBJ           0   /*Check the object's type and existence (e.g. not deleted). (Slow)*/

/*Add a custom handler when assert happens e.g. to restart the MCU*/
#define LV_ASSERT_HANDLER_INCLUDE <stdint.h>
#define LV_ASSERT_HANDLER while(1);   /*Halt by default*/

/*-------------
 * Others
 *-----------*/

/*1: Show CPU usage and FPS count*/
#define LV_USE_PERF_MONITOR 1
#if LV_USE_PERF_MONITOR
    #define LV_USE_PERF_MONITOR_POS LV_ALIGN_BOTTOM_RIGHT
#endif

/*1: Show the used memory and the memory fragmentation
 * Requires LV_MEM_CUSTOM = 0*/
#define LV_USE_MEM_MONITOR 0
#if LV_USE_MEM_MONITOR
    #define LV_USE_MEM_MONITOR_POS LV_ALIGN_BOTTOM_LEFT
#endif

```

```c
/*1: Draw random colored rectangles over the redrawn areas*/
#define LV_USE_REFR_DEBUG 0

/*Change the built in (v)snprintf functions*/
#define LV_SPRINTF_CUSTOM 0
#if LV_SPRINTF_CUSTOM
    #define LV_SPRINTF_INCLUDE <stdio.h>
    #define lv_snprintf  snprintf
    #define lv_vsnprintf vsnprintf
#else   /*LV_SPRINTF_CUSTOM*/
    #define LV_SPRINTF_USE_FLOAT 0
#endif  /*LV_SPRINTF_CUSTOM*/

#define LV_USE_USER_DATA 1

/*Garbage Collector settings
 *Used if lvgl is bound to higher level language and the memory is managed by that language*/
#define LV_ENABLE_GC 0
#if LV_ENABLE_GC != 0
    #define LV_GC_INCLUDE "gc.h"                           /*Include Garbage Collector related things*/
#endif /*LV_ENABLE_GC*/

/*=====================
 *  COMPILER SETTINGS
 *====================*/

/*For big endian systems set to 1*/
#define LV_BIG_ENDIAN_SYSTEM 0

/*Define a custom attribute to `lv_tick_inc` function*/
#define LV_ATTRIBUTE_TICK_INC

/*Define a custom attribute to `lv_timer_handler` function*/
#define LV_ATTRIBUTE_TIMER_HANDLER

/*Define a custom attribute to `lv_disp_flush_ready` function*/
#define LV_ATTRIBUTE_FLUSH_READY

/*Required alignment size for buffers*/
#define LV_ATTRIBUTE_MEM_ALIGN_SIZE 1

/*Will be added where memories needs to be aligned (with -Os data might not be aligned to boundary by default).
 * E.g. __attribute__((aligned(4)))*/
#define LV_ATTRIBUTE_MEM_ALIGN
```

```
/*Attribute to mark large constant arrays for example font's bitmaps*/
#define LV_ATTRIBUTE_LARGE_CONST

/*Compiler prefix for a big array declaration in RAM*/
#define LV_ATTRIBUTE_LARGE_RAM_ARRAY

/*Place performance critical functions into a faster memory (e.g RAM)*/
#define LV_ATTRIBUTE_FAST_MEM

/*Prefix variables that are used in GPU accelerated operations, often these need to be placed in RAM sections that are DMA accessible*/
#define LV_ATTRIBUTE_DMA

/*Export integer constant to binding. This macro is used with constants in the form of LV_<CONST> that
 *should also appear on LVGL binding API such as Micropython.*/
#define LV_EXPORT_CONST_INT(int_value) struct _silence_gcc_warning /*The default value just prevents GCC warning*/

/*Extend the default -32k..32k coordinate range to -4M..4M by using int32_t for coordinates instead of int16_t*/
#define LV_USE_LARGE_COORD 0

/*==================
 *   FONT USAGE
 *===================*/

/*Montserrat fonts with ASCII range and some symbols using bpp = 4
 *https://fonts.google.com/specimen/Montserrat*/
#define LV_FONT_MONTSERRAT_8  0
#define LV_FONT_MONTSERRAT_10 0
#define LV_FONT_MONTSERRAT_12 0
#define LV_FONT_MONTSERRAT_14 1
#define LV_FONT_MONTSERRAT_16 0
#define LV_FONT_MONTSERRAT_18 0
#define LV_FONT_MONTSERRAT_20 0
#define LV_FONT_MONTSERRAT_22 0
#define LV_FONT_MONTSERRAT_24 0
#define LV_FONT_MONTSERRAT_26 0
#define LV_FONT_MONTSERRAT_28 0
#define LV_FONT_MONTSERRAT_30 0
#define LV_FONT_MONTSERRAT_32 0
#define LV_FONT_MONTSERRAT_34 0
#define LV_FONT_MONTSERRAT_36 0
#define LV_FONT_MONTSERRAT_38 0
#define LV_FONT_MONTSERRAT_40 0
```

```c
#define LV_FONT_MONTSERRAT_42 0
#define LV_FONT_MONTSERRAT_44 0
#define LV_FONT_MONTSERRAT_46 0
#define LV_FONT_MONTSERRAT_48 0

/*Demonstrate special features*/
#define LV_FONT_MONTSERRAT_12_SUBPX      0
#define LV_FONT_MONTSERRAT_28_COMPRESSED 0  /*bpp = 3*/
#define LV_FONT_DEJAVU_16_PERSIAN_HEBREW 0  /*Hebrew, Arabic, Persian letters and all their forms*/
#define LV_FONT_SIMSUN_16_CJK            0  /*1000 most common CJK radicals*/

/*Pixel perfect monospace fonts*/
#define LV_FONT_UNSCII_8  0
#define LV_FONT_UNSCII_16 0

/*Optionally declare custom fonts here.
 *You can use these fonts as default font too and they will be available globally.
 *E.g. #define LV_FONT_CUSTOM_DECLARE   LV_FONT_DECLARE(my_font_1) LV_FONT_DECLARE(my_font_2)*/
#define LV_FONT_CUSTOM_DECLARE

/*Always set a default font*/
#define LV_FONT_DEFAULT &lv_font_montserrat_14

/*Enable handling large font and/or fonts with a lot of characters.
 *The limit depends on the font size, font face and bpp.
 *Compiler error will be triggered if a font needs it.*/
#define LV_FONT_FMT_TXT_LARGE 0

/*Enables/disables support for compressed fonts.*/
#define LV_USE_FONT_COMPRESSED 0

/*Enable subpixel rendering*/
#define LV_USE_FONT_SUBPX 0
#if LV_USE_FONT_SUBPX
    /*Set the pixel order of the display. Physical order of RGB channels. Doesn't matter with "normal" fonts.*/
    #define LV_FONT_SUBPX_BGR 0  /*0: RGB; 1:BGR order*/
#endif

/*Enable drawing placeholders when glyph dsc is not found*/
#define LV_USE_FONT_PLACEHOLDER 1

/*=================
 *  TEXT SETTINGS
```

```c
 *=================*/

/**
 * Select a character encoding for strings.
 * Your IDE or editor should have the same character encoding
 * - LV_TXT_ENC_UTF8
 * - LV_TXT_ENC_ASCII
 */
#define LV_TXT_ENC LV_TXT_ENC_UTF8

/*Can break (wrap) texts on these chars*/
#define LV_TXT_BREAK_CHARS " ,.;:-_"

/*If a word is at least this long, will break wherever "prettiest"
 *To disable, set to a value <= 0*/
#define LV_TXT_LINE_BREAK_LONG_LEN 0

/*Minimum number of characters in a long word to put on a line before a break.
 *Depends on LV_TXT_LINE_BREAK_LONG_LEN.*/
#define LV_TXT_LINE_BREAK_LONG_PRE_MIN_LEN 3

/*Minimum number of characters in a long word to put on a line after a break.
 *Depends on LV_TXT_LINE_BREAK_LONG_LEN.*/
#define LV_TXT_LINE_BREAK_LONG_POST_MIN_LEN 3

/*The control character to use for signalling text recoloring.*/
#define LV_TXT_COLOR_CMD "#"

/*Support bidirectional texts. Allows mixing Left-to-Right and Right-to-Left texts.
 *The direction will be processed according to the Unicode Bidirectional Algorithm:
 *https://www.w3.org/International/articles/inline-bidi-markup/uba-basics*/
#define LV_USE_BIDI 0
#if LV_USE_BIDI
    /*Set the default direction. Supported values:
    *`LV_BASE_DIR_LTR` Left-to-Right
    *`LV_BASE_DIR_RTL` Right-to-Left
    *`LV_BASE_DIR_AUTO` detect texts base direction*/
    #define LV_BIDI_BASE_DIR_DEF LV_BASE_DIR_AUTO
#endif

/*Enable Arabic/Persian processing
 *In these languages characters should be replaced with an other form based on their position in the text*/
#define LV_USE_ARABIC_PERSIAN_CHARS 0

```

```c
/*==================
 *  WIDGET USAGE
 *================*/

/*Documentation of the widgets: https://docs.lvgl.io/latest/en/html/widgets/index.html*/

#define LV_USE_ARC        1

#define LV_USE_BAR        1

#define LV_USE_BTN        1

#define LV_USE_BTNMATRIX  1

#define LV_USE_CANVAS     1

#define LV_USE_CHECKBOX   1

#define LV_USE_DROPDOWN   1   /*Requires: lv_label*/

#define LV_USE_IMG        1   /*Requires: lv_label*/

#define LV_USE_LABEL      1
#if LV_USE_LABEL
    #define LV_LABEL_TEXT_SELECTION 1 /*Enable selecting text of the label*/
    #define LV_LABEL_LONG_TXT_HINT 1  /*Store some extra info in labels to speed up drawing of very long texts*/
#endif

#define LV_USE_LINE       1

#define LV_USE_ROLLER     1   /*Requires: lv_label*/
#if LV_USE_ROLLER
    #define LV_ROLLER_INF_PAGES 7 /*Number of extra "pages" when the roller is infinite*/
#endif

#define LV_USE_SLIDER     1   /*Requires: lv_bar*/

#define LV_USE_SWITCH     1

#define LV_USE_TEXTAREA   1   /*Requires: lv_label*/
#if LV_USE_TEXTAREA != 0
    #define LV_TEXTAREA_DEF_PWD_SHOW_TIME 1500    /*ms*/
#endif

```

```c
#define LV_USE_TABLE      1

/*==================
 * EXTRA COMPONENTS
 *==================*/

/*-----------
 * Widgets
 *----------*/
#define LV_USE_ANIMIMG    1

#define LV_USE_CALENDAR   1
#if LV_USE_CALENDAR
    #define LV_CALENDAR_WEEK_STARTS_MONDAY 0
    #if LV_CALENDAR_WEEK_STARTS_MONDAY
        #define LV_CALENDAR_DEFAULT_DAY_NAMES {"Mo", "Tu", "We", "Th", "Fr", "Sa", "Su"}
    #else
        #define LV_CALENDAR_DEFAULT_DAY_NAMES {"Su", "Mo", "Tu", "We", "Th", "Fr", "Sa"}
    #endif

    #define LV_CALENDAR_DEFAULT_MONTH_NAMES {"January", "February", "March", "April", "May",  "June", "July", "August", "September", "October", "November", "December"}
    #define LV_USE_CALENDAR_HEADER_ARROW 1
    #define LV_USE_CALENDAR_HEADER_DROPDOWN 1
#endif  /*LV_USE_CALENDAR*/

#define LV_USE_CHART      1

#define LV_USE_COLORWHEEL 1

#define LV_USE_IMGBTN     1

#define LV_USE_KEYBOARD   1

#define LV_USE_LED        1

#define LV_USE_LIST       1

#define LV_USE_MENU       1

#define LV_USE_METER      1

#define LV_USE_MSGBOX     1

```

```c
#define LV_USE_SPAN       1
#if LV_USE_SPAN
    /*A line text can contain maximum num of span descriptor */
    #define LV_SPAN_SNIPPET_STACK_SIZE 64
#endif

#define LV_USE_SPINBOX    1

#define LV_USE_SPINNER    1

#define LV_USE_TABVIEW    1

#define LV_USE_TILEVIEW   1

#define LV_USE_WIN        1

/*-----------
 * Themes
 *----------*/

/*A simple, impressive and very complete theme*/
#define LV_USE_THEME_DEFAULT 1
#if LV_USE_THEME_DEFAULT

    /*0: Light mode; 1: Dark mode*/
    #define LV_THEME_DEFAULT_DARK 0

    /*1: Enable grow on press*/
    #define LV_THEME_DEFAULT_GROW 1

    /*Default transition time in [ms]*/
    #define LV_THEME_DEFAULT_TRANSITION_TIME 80
#endif /*LV_USE_THEME_DEFAULT*/

/*A very simple theme that is a good starting point for a custom theme*/
#define LV_USE_THEME_BASIC 1

/*A theme designed for monochrome displays*/
#define LV_USE_THEME_MONO 1

/*-----------
 * Layouts
 *----------*/

/*A layout similar to Flexbox in CSS.*/
#define LV_USE_FLEX 1

```

```
/*A layout similar to Grid in CSS.*/
#define LV_USE_GRID 1

/*---------------------
 * 3rd party libraries
 *--------------------*/

/*File system interfaces for common APIs */

/*API for fopen, fread, etc*/
#define LV_USE_FS_STDIO 0
#if LV_USE_FS_STDIO
    #define LV_FS_STDIO_LETTER '\0'     /*Set an upper cased letter on which the drive will accessible (e.g. 'A')*/
    #define LV_FS_STDIO_PATH ""         /*Set the working directory. File/directory paths will be appended to it.*/
    #define LV_FS_STDIO_CACHE_SIZE 0    /*>0 to cache this number of bytes in lv_fs_read()*/
#endif

/*API for open, read, etc*/
#define LV_USE_FS_POSIX 0
#if LV_USE_FS_POSIX
    #define LV_FS_POSIX_LETTER '\0'     /*Set an upper cased letter on which the drive will accessible (e.g. 'A')*/
    #define LV_FS_POSIX_PATH ""         /*Set the working directory. File/directory paths will be appended to it.*/
    #define LV_FS_POSIX_CACHE_SIZE 0    /*>0 to cache this number of bytes in lv_fs_read()*/
#endif

/*API for CreateFile, ReadFile, etc*/
#define LV_USE_FS_WIN32 0
#if LV_USE_FS_WIN32
    #define LV_FS_WIN32_LETTER '\0'     /*Set an upper cased letter on which the drive will accessible (e.g. 'A')*/
    #define LV_FS_WIN32_PATH ""         /*Set the working directory. File/directory paths will be appended to it.*/
    #define LV_FS_WIN32_CACHE_SIZE 0    /*>0 to cache this number of bytes in lv_fs_read()*/
#endif

/*API for FATFS (needs to be added separately). Uses f_open, f_read, etc*/
#define LV_USE_FS_FATFS 0
#if LV_USE_FS_FATFS
    #define LV_FS_FATFS_LETTER '\0'     /*Set an upper cased letter on which the drive will accessible (e.g. 'A')*/
```

```
    #define LV_FS_FATFS_CACHE_SIZE 0    /*>0 to cache this number of bytes in lv_fs_read()*/
#endif

/*API for LittleFS (library needs to be added separately). Uses lfs_file_open, lfs_file_read, etc*/
#define LV_USE_FS_LITTLEFS 0
#if LV_USE_FS_LITTLEFS
    #define LV_FS_LITTLEFS_LETTER '\0'     /*Set an upper cased letter on which the drive will accessible (e.g. 'A')*/
    #define LV_FS_LITTLEFS_CACHE_SIZE 0    /*>0 to cache this number of bytes in lv_fs_read()*/
#endif

/*PNG decoder library*/
#define LV_USE_PNG 0

/*BMP decoder library*/
#define LV_USE_BMP 0

/* JPG + split JPG decoder library.
 * Split JPG is a custom format optimized for embedded systems. */
#define LV_USE_SJPG 0

/*GIF decoder library*/
#define LV_USE_GIF 0

/*QR code library*/
#define LV_USE_QRCODE 0

/*FreeType library*/
#define LV_USE_FREETYPE 0
#if LV_USE_FREETYPE
    /*Memory used by FreeType to cache characters [bytes] (-1: no caching)*/
    #define LV_FREETYPE_CACHE_SIZE (16 * 1024)
    #if LV_FREETYPE_CACHE_SIZE >= 0
        /* 1: bitmap cache use the sbit cache, 0:bitmap cache use the image cache. */
        /* sbit cache:it is much more memory efficient for small bitmaps(font size < 256) */
        /* if font size >= 256, must be configured as image cache */
        #define LV_FREETYPE_SBIT_CACHE 0
        /* Maximum number of opened FT_Face/FT_Size objects managed by this cache instance. */
        /* (0:use system defaults) */
        #define LV_FREETYPE_CACHE_FT_FACES 0
        #define LV_FREETYPE_CACHE_FT_SIZES 0
```

```
    #endif
#endif

/*Tiny TTF library*/
#define LV_USE_TINY_TTF 0
#if LV_USE_TINY_TTF
    /*Load TTF data from files*/
    #define LV_TINY_TTF_FILE_SUPPORT 0
#endif

/*Rlottie library*/
#define LV_USE_RLOTTIE 0

/*FFmpeg library for image decoding and playing videos
 *Supports all major image formats so do not enable other image decoder with it*/
#define LV_USE_FFMPEG 0
#if LV_USE_FFMPEG
    /*Dump input information to stderr*/
    #define LV_FFMPEG_DUMP_FORMAT 0
#endif

/*-----------
 * Others
 *----------*/

/*1: Enable API to take snapshot for object*/
#define LV_USE_SNAPSHOT 0

/*1: Enable Monkey test*/
#define LV_USE_MONKEY 0

/*1: Enable grid navigation*/
#define LV_USE_GRIDNAV 0

/*1: Enable lv_obj fragment*/
#define LV_USE_FRAGMENT 0

/*1: Support using images as font in label or span widgets */
#define LV_USE_IMGFONT 0

/*1: Enable a published subscriber based messaging system */
#define LV_USE_MSG 0

/*1: Enable Pinyin input method*/
/*Requires: lv_keyboard*/
#define LV_USE_IME_PINYIN 0
```

```
#if LV_USE_IME_PINYIN
    /*1: Use default thesaurus*/
    /*If you do not use the default thesaurus, be sure to use `lv_ime_pinyin`
after setting the thesauruss*/
    #define LV_IME_PINYIN_USE_DEFAULT_DICT 1
    /*Set the maximum number of candidate panels that can be displayed*/
    /*This needs to be adjusted according to the size of the screen*/
    #define LV_IME_PINYIN_CAND_TEXT_NUM 6

    /*Use 9 key input(k9)*/
    #define LV_IME_PINYIN_USE_K9_MODE      1
    #if LV_IME_PINYIN_USE_K9_MODE == 1
        #define LV_IME_PINYIN_K9_CAND_TEXT_NUM 3
    #endif // LV_IME_PINYIN_USE_K9_MODE
#endif

/*==================
* EXAMPLES
*==================*/

/*Enable the examples to be built with the library*/
#define LV_BUILD_EXAMPLES 1

/*===================
 * DEMO USAGE
 ====================*/

/*Show some widget. It might be required to increase `LV_MEM_SIZE` */
#define LV_USE_DEMO_WIDGETS 0
#if LV_USE_DEMO_WIDGETS
#define LV_DEMO_WIDGETS_SLIDESHOW 0
#endif

/*Demonstrate the usage of encoder and keyboard*/
#define LV_USE_DEMO_KEYPAD_AND_ENCODER 0

/*Benchmark your system*/
#define LV_USE_DEMO_BENCHMARK 1
#if LV_USE_DEMO_BENCHMARK
/*Use RGB565A8 images with 16 bit color depth instead of ARGB8565*/
#define LV_DEMO_BENCHMARK_RGB565A8 0
#endif

/*Stress test for LVGL*/
#define LV_USE_DEMO_STRESS 0

/*Music player demo*/
```

```c
#define LV_USE_DEMO_MUSIC 0
#if LV_USE_DEMO_MUSIC
    #define LV_DEMO_MUSIC_SQUARE    0
    #define LV_DEMO_MUSIC_LANDSCAPE 0
    #define LV_DEMO_MUSIC_ROUND     0
    #define LV_DEMO_MUSIC_LARGE     0
    #define LV_DEMO_MUSIC_AUTO_PLAY 0
#endif

/*--END OF LV_CONF_H--*/

#endif /*LV_CONF_H*/

#endif /*End of "Content enable"*/

```